

Scaling and Fault Tolerance Strategy

1. System Component Scaling

To handle millions of concurrent users during blockbuster ticket releases without degraded latency or system crashes, each layer scales independently:

Horizontal Scaling of Application Servers

- Microservices (`BookingService`, `CatalogService`, `NotificationService`) are designed strictly stateless.
- Instances run behind an Application Load Balancer (ALB) distributing traffic using the least-outstanding-requests algorithm.
- Auto-scalers dynamically scale instances out and in based on CPU utilization (> 70%) and request latency thresholds.

Database Sharding and Replication (PostgreSQL)

- **Primary-Replica Topology:** All transactional writes (`bookings`, `payments`, `seat_locks`) hit the Primary database. Read-heavy browsing traffic (`movies`, `theaters`, `shows`) is offloaded across multiple asynchronous Read Replicas.
- **Horizontal Sharding by City:** The database is horizontally sharded by `city_id`. Because movie browsing and ticket bookings are strictly localized to a user's chosen city, queries and transactions execute entirely within a single database shard, avoiding distributed cross-shard transactions.

Partitioned Message Queues (Kafka)

- The high-throughput `booking_events` topic is divided into multiple partitions using `show_id` as the partition key.
- Downstream consumer instances scale horizontally to match the number of partitions, allowing parallel consumption while maintaining strict per-show event ordering.

2. Fault Tolerance & Resilience Mechanisms

Circuit Breakers for External Integrations

External payment processors and SMS gateways represent the highest latency and failure risk. All outbound external calls are protected with circuit breakers:

- If failure rates exceed 50% over a 30-second rolling window, the circuit trips to **OPEN**.
- The system fails fast and automatically diverts checkout requests to a pre-configured secondary payment gateway, preventing thread exhaustion on application servers.

Graceful Degradation

Under extreme peak traffic, non-essential services are shed:

- Resource-intensive background features (personalized movie recommendations, trending user reviews, promotional banners) are bypassed.
- Compute capacity, database connections, and cache bandwidth are reserved exclusively for the core seat reservation and checkout flow.

Auto-Retry with Exponential Backoff

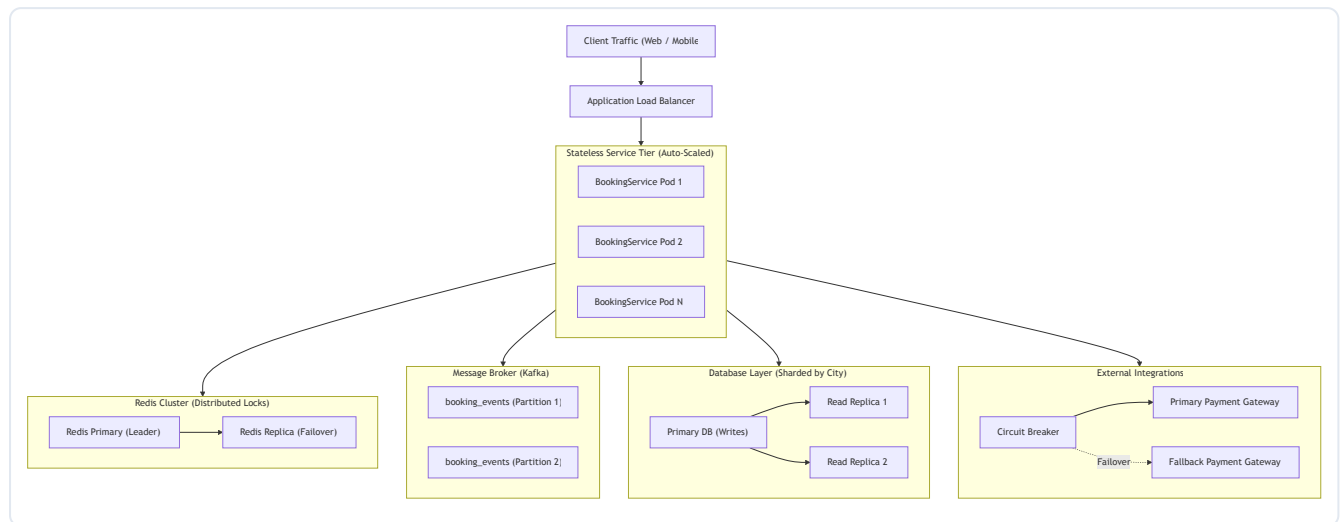
- Internal inter-service requests experiencing transient network timeouts retry automatically up to 3 times with exponential backoff and randomized jitter (50ms, 100ms, 200ms).
- Jitter prevents the "thundering herd" problem from overwhelming recovering services.

Quorum Reads and Writes

- In distributed data systems (Kafka brokers and Redis replication), writes require acknowledgment from a quorum of in-sync replicas (`acks=all`, `min.insync.replicas=2`).
- This guarantees zero data loss and prevents split-brain anomalies if an active leader node experiences sudden hardware failure.

3. Architecture Diagram: Scaling & Fault Tolerance

The architecture diagram below highlights the multi-tier scaling and resilience layers designed to eliminate single points of failure across all services:



Architectural Tier Breakdown & Interactions

- **Ingress Tier:** The Application Load Balancer terminates SSL, enforces rate limiting, and routes requests to the stateless service tier.
- **Stateless Service Tier:** Containerized BookingService instances autoscale horizontally based on real-time traffic spikes, allowing zero-downtime rolling updates.
- **Caching & Locking Tier:** Redis Primary-Replica cluster handles sub-millisecond seat locks (120s TTL) and caches hot show schedules, eliminating read bottlenecks on PostgreSQL.
- **Event Streaming Tier:** Partitioned Kafka topics decouple non-blocking downstream processes (notifications, analytics, loyalty rewards) from synchronous booking completion.
- **Data Tier:** PostgreSQL Primary instances handle ACID booking and payment writes partitioned by `city_id`, while multiple Read Replicas serve catalog queries.
- **Resilience Boundary:** Circuit breakers wrap third-party payment gateways, instantly failing over to backup payment processors without thread exhaustion.